# Aplicacion de gestión

# Principios de diseño (para evitar duplicación con Moodle)

1. **Moodle como LMS "System of Record"** para: cursos, estructura de curso, actividades, libro de calificaciones operativo, matriculación en curso (enrolments), usuarios base.
2. **Aplicación complementaria como "Academic Admin System"** para:
    - Reinscripción por período (workflow administrativo)
    - Reportería institucional (por período, cohorte, comparativos, consolidaciones)
    - Generación de récord oficial (PDF con firma/validación)
    - Consolidación histórica multiperíodo y trazabilidad/auditoría
3. **Modelo "Mirror + Extend"**: se mantiene una **copia espejo** (cache operacional) de entidades Moodle necesarias para reportar, y se agregan entidades propias (períodos, solicitudes, récord oficial, auditoría, etc.).

---

# 1) Arquitectura técnica (alto nivel)

## 1.1 Componentes (recomendación: "Modular Monolith + Workers" o microservicios ligeros)

**Opción recomendada (pragmática): Modular Monolith** con colas y workers:

- **API Backend (Core)**
    - Auth & RBAC
    - Módulo Académico (períodos, reinscripción, historial)
    - Módulo Calificaciones (captura local, validación, push a Moodle)
    - Reportes (consultas agregadas)
    - Generación de Documentos (récord oficial PDF)
- **Integration Service (Moodle Adapter)**
    - Cliente REST hacia Moodle Web Services
    - Normalización y mapeo de IDs Moodle ↔ App
- **Sync Engine (Workers)**

    - Consumo de eventos (webhooks) y/o polling incremental
    - Reintentos, DLQ (dead-letter queue), reconciliación
- **Message Broker** (RabbitMQ / Azure Service Bus / Kafka)
    - Topología: `sync.inbound`, `sync.outbound`, `sync.reconcile`, `audit`
- **DB**: PostgreSQL (recomendado) + Redis (cache / locks)
- **Observabilidad**: logs estructurados, métricas, trazas (OpenTelemetry)

## 1.2 Diagrama lógico (texto)

```
[Web/Mobile UI]
     |
     v
[API Gateway / Backend Core] -----> [PostgreSQL]
     |                      \-----> [Redis]
     |
     +--> [Integration: Moodle Adapter] --REST--> [Moodle Web Services]
     |
     +--> [Outbox -> Message Broker] <--> [Sync Workers]
                                ^
                                |
                      [Moodle Webhooks/Plugin] --POST-->
[/integrations/moodle/webhook]
```

Moodle Web Services usa token por usuario/servicio y permite configurar protocolos (REST) y capacidades asociadas a funciones. [docs.moodle.org], [moodledev.io]

---

# 2) (1) Estructura de módulos funcionales

## 2.1 Módulos requeridos (según tu lista) y su responsabilidad

1. **Emulación de estructura Moodle (Mirror)**

- Sincroniza y almacena localmente:
    - Cursos
    - Usuarios (estudiantes/profesores)
    - Matrículas (enrolments)
    - Elementos de calificación (grade items) y calificaciones relevantes
- **No reemplaza** Moodle, solo sirve para consultas rápidas, reportes y consolidación.

2. **Registro de calificaciones (captura y almacenamiento)**

- Interfaz docente para registrar notas (por curso)
- Valida reglas (rango, ponderaciones, cierre de período)
- Persiste en DB local + dispara evento “outbound” hacia Moodle.

3. **Reporte de notas**

- Vistas por estudiante / curso / período/programa
- Incluye filtros y exportaciones (PDF/Excel)
- Se alimenta del “mirror” y de tablas históricas.

4. **Récord de notas (historial académico)**

- Consolidación por períodos/ciclos
- Incluye equivalencias, retiros, repetición, etc. (si aplica)

5. **Reporte por estudiante (detalle individual)**

- Tablero del estudiante: cursos, calificaciones, progreso histórico
- Descarga de récord (documento oficial o reporte interno)

6. **Gestión de reinscripción (matrícula nuevos períodos)**

- Workflow: preselección → validaciones (deudas, requisitos) → confirmación → creación/actualización de enrolments en Moodle
- Estados: borrador, pendiente, aprobado, rechazado, aplicado

7. **Generación de récord de notas (documento oficial)**

- Generación de PDF con: datos del alumno, períodos, cursos, calificaciones finales, promedio
- Firma/validación: hash/QR con verificación pública opcional

Cursos disponibles

---

# 3) (2) Flujos de comunicación bidireccional con Moodle

## 3.1 Canales de integración

### A) App → Moodle (Outbound)

- Usando **Moodle Web Services REST** (server.php) con `wstoken` y `wsfunction`, configurado mediante un **External Service** con funciones limitadas. [docs.moodle.org], [moodledev.io]

Ejemplos típicos de outbound:

- Crear/actualizar enrolments (por reinscripción)
- Actualizar calificaciones (cuando el profesor captura en la app)
- (Opcional) crear usuarios si el proceso de admisiones lo exige

**B) Moodle → App (Inbound)**
Dos estrategias combinables:

1. **Webhooks / Plugin de eventos** (casi real-time)

- Moodle envía **POST** al callback de la app con evento + payload o referencias, evitando polling continuo. [moodle.org]

2. **Polling incremental** (fallback / reconciliación)

- Jobs programados que consultan Moodle por cambios desde "última marca de agua" (timestamp/modified/grade history si está disponible)
- Útil cuando no se instala plugin o hay eventos que no llegan

## 3.2 Flujos por dominio (secuencias)

### Flujo 1: Sincronización inicial (bootstrap)

1. Admin registra credenciales (URL Moodle, token servicio, protocolos)
2. App ejecuta importación:
    - Cursos
    - Usuarios (estudiantes/profesores)
    - Matrículas (enrolments)
    - Estructura de calificaciones (grade items) y calificaciones base
3. Se almacenan mapeos `moodle_id ↔ app_id`
4. Se crea "checkpoint" de sincronización

El concepto de enrolment en Moodle es distinto de role assignment; enrolments viven en `user_enrolments` y roles en `role_assignments`, por lo que tu sincronización debe considerar ambas dimensiones. [docs.moodle.org]

### Flujo 2: Profesor registra nota en la App → Moodle

1. Profesor guarda calificación en UI
2. Backend valida + guarda en `grades` (local)
3. Se escribe evento en **Outbox** (transaccional)
4. Worker lee Outbox y llama WS de Moodle para actualizar calificación

5. Moodle responde OK / error → se marca estado y se audita

### Flujo 3: Nota cambiada en Moodle → App (bidireccional)

1. Moodle dispara webhook `grade_updated` (o equivalente)
2. App recibe `/integrations/moodle/webhook`
3. Encola "fetch details" (evitar confiar en payload incompleto)
4. Worker consulta Moodle por detalles de la nota y actualiza mirror/local
5. Se registra auditoría + control de conflictos

### Flujo 4: Reinscripción (App) → enrolments en Moodle

1. Admin configura período académico en la app
2. Estudiante solicita reinscripción (o admin procesa lote)
3. App valida reglas internas
4. Al aprobar, app crea/actualiza enrollments en Moodle (manual enrolment / cohort / método definido)
5. App actualiza estado "aplicado" y refleja matrícula local

---

# 4) (3) Modelos de datos y esquema de base de datos (propuesta)

## 4.1 Enfoque de datos

- **Entidades espejo (Mirror)**: guardan lo mínimo necesario para reportes y operaciones complementarias.
- **Entidades propias (Extend)**: períodos, reinscripción, récord oficial, auditoría, etc.
- **Mapping**: tabla de mapeo para IDs Moodle y control de versiones.

## 4.2 Entidades principales

### Seguridad / acceso

- `app_users` (cuenta en la app; puede estar vinculada a un moodle_user)
- `roles` (STUDENT, TEACHER, ADMIN)
- `user_roles` (N:M)
- `permissions` (opcional granular)
- `sessions` / `refresh_tokens` (si aplica)

### Mirror Moodle

- `moodle_users` (id Moodle, username, email, nombre, tipo: student/teacher, estado)
- `moodle_courses` (id Moodle, fullname, shortname, category, visible, fechas)
- `moodle_enrolments` (course_id, moodle_user_id, role, status, start/end)
- `moodle_grade_items` (course_id, itemid, itemname, itemtype, grademax, weight, etc.)
- `moodle_grades` (grade_item_id, moodle_user_id, grade, feedback, timemodified)

### Académico (propio)

- `academic_periods` (período, fechas, estado)
- `re_enrolment_requests` (solicitud, estudiante, período, estado, metadata)
- `re_enrolment_lines` (cursos solicitados/confirmados)
- `local_grades` (registro "operacional" en la app; fuente si el profesor captura aquí)
- `student_records` (historial consolidado por período)
- `transcripts` (récord oficial generado: número, hash, url, estado)
- `audit_log` (quién cambió qué, cuándo, antes/después)
- `sync_state` (marcas de agua por entidad)
- `sync_jobs` / `dead_letter` (errores y reintentos)

## 4.3 Esquema SQL (PostgreSQL – ejemplo base)

SQL

```
-- Roles
create table roles (
id bigserial primary key,
code text unique not null, -- STUDENT, TEACHER, ADMIN
description text
);

create table app_users (
id bigserial primary key,
email text unique not null,
password_hash text not null,
display_name text,
moodle_user_id bigint null, -- vínculo opcional
is_active boolean not null default true,
created_at timestamptz not null default now()
);

create table user_roles (
user_id bigint not null references app_users(id) on delete cascade,
role_id bigint not null references roles(id) on delete cascade,
primary key (user_id, role_id)
);
```

```
-- Mirror Moodle
create table moodle_users (
id bigserial primary key,
moodle_id bigint unique not null,
username text,
email text,
firstname text,
lastname text,
user_type text not null, -- student|teacher|other
suspended boolean not null default false,
timemodified timestamptz,
last_synced_at timestamptz
);

create table moodle_courses (
id bigserial primary key,
moodle_id bigint unique not null,
shortname text,
fullname text,
category_id bigint,
visible boolean,
startdate timestamptz,
enddate timestamptz,
timemodified timestamptz,
last_synced_at timestamptz
);

create table moodle_enrolments (
id bigserial primary key,
course_moodle_id bigint not null,
user_moodle_id bigint not null,
role_code text not null, -- student|teacher
enrol_status text not null, -- active|suspended
timestart timestamptz,
timeend timestamptz,
timemodified timestamptz,
unique (course_moodle_id, user_moodle_id, role_code)
);

create table moodle_grade_items (
id bigserial primary key,
course_moodle_id bigint not null,
```

```
item_moodle_id bigint not null,
itemname text,
itemtype text,
grademax numeric(10,3),
grademin numeric(10,3),
weight numeric(10,6),
timemodified timestamptz,
unique (course_moodle_id, item_moodle_id)
);

create table moodle_grades (
id bigserial primary key,
course_moodle_id bigint not null,
item_moodle_id bigint not null,
user_moodle_id bigint not null,
grade numeric(10,3),
feedback text,
timemodified timestamptz,
last_synced_at timestamptz,
unique (course_moodle_id, item_moodle_id, user_moodle_id)
);

-- Académico propio
create table academic_periods (
id bigserial primary key,
code text unique not null, -- 2026-01, 2026-Q1, etc.
name text not null,
start_date date not null,
end_date date not null,
status text not null, -- planned|open|closed
created_at timestamptz not null default now()
);

create table re_enrolment_requests (
id bigserial primary key,
app_user_id bigint not null references app_users(id),
period_id bigint not null references academic_periods(id),
status text not null, -- draft|submitted|approved|rejected|applied
submitted_at timestamptz,
decided_at timestamptz,
decided_by bigint references app_users(id),
notes text
);
```

```
create table re_enrolment_lines (
id bigserial primary key,
request_id bigint not null references re_enrolment_requests(id) on delete cascade,
course_moodle_id bigint not null,
action text not null, -- enrol|suspend|unenrol (según política)
status text not null default 'pending', -- pending|done|failed
last_error text
);

-- Registro local (cuando el profesor captura en la app)
create table local_grades (
id bigserial primary key,
course_moodle_id bigint not null,
item_moodle_id bigint not null,
student_moodle_id bigint not null,
teacher_app_user_id bigint not null references app_users(id),
grade numeric(10,3) not null,
feedback text,
source text not null, -- app|moodle
version int not null default 1,
updated_at timestamptz not null default now(),
unique (course_moodle_id, item_moodle_id, student_moodle_id)
);

-- Outbox para sincronización confiable
create table outbox_events (
id bigserial primary key,
event_type text not null,
payload jsonb not null,
status text not null default 'pending', -- pending|processing|done|failed
attempts int not null default 0,
next_attempt_at timestamptz not null default now(),
created_at timestamptz not null default now(),
last_error text
);

create table sync_state (
id bigserial primary key,
entity text unique not null, -- users|courses|enrolments|grades|grade_items
last_checkpoint text, -- puede ser timestamp/offset
updated_at timestamptz not null default now()
);
```

```
create table audit_log (
id bigserial primary key,
actor_user_id bigint references app_users(id),
action text not null,
entity text not null,
entity_key text not null,
before_data jsonb,
after_data jsonb,
created_at timestamptz not null default now()
);

create table transcripts (
id bigserial primary key,
app_user_id bigint not null references app_users(id),
period_id bigint references academic_periods(id),
transcript_number text unique not null,
file_url text,
file_hash text not null,
status text not null, -- generated|revoked
generated_at timestamptz not null default now()
);
Show more lines
```

El componente de calificaciones en Moodle incluye APIs/funcionalidades de gradebook (clases externas, funciones y utilidades relacionadas con "grades"), que normalmente se consumen vía web services específicos y/o endpoints externos. [wimski.org]

---

# 5) (4) Especificación de endpoints / servicios requeridos (App)

Los siguientes endpoints se asumen **REST JSON**, con JWT/OAuth2 (o cookies si web interna). Versiona: `/api/v1`.

## 5.1 Autenticación / usuarios

- `POST /auth/login`
- `POST /auth/refresh`
- `POST /auth/logout`
- `GET /me`

- `GET /users` (admin)
- `POST /users` (admin)
- `PATCH /users/{id}` (admin)
- `GET /users/{id}/roles` (admin)
- `PUT /users/{id}/roles` (admin)

## 5.2 Estructura académica (mirror + periodos)

- `GET /courses` (admin/teacher/student según contexto)
- `GET /courses/{courseId}`
- `GET /courses/{courseId}/students` (teacher/admin)
- `GET /courses/{courseId}/grade-items` (teacher/admin)
- `GET /periods` (admin)
- `POST /periods` (admin)
- `PATCH /periods/{id}` (admin)
- `POST /periods/{id}/close` (admin)

## 5.3 Registro y consulta de calificaciones

- `POST /grades` (teacher)
  Payload: `{course_moodle_id, item_moodle_id, student_moodle_id, grade, feedback}`
- `PATCH /grades/{id}` (teacher)
- `GET /grades` (filtros: estudiante/curso/período)
- `GET /students/{studentId}/grades` (student/teacher/admin con reglas)

## 5.4 Reportes

- `GET /reports/grades/by-student/{studentId}`
- `GET /reports/grades/by-course/{courseId}`
- `GET /reports/grades/by-period/{periodId}` (admin)
- `GET /reports/global/summary` (admin)

## 5.5 Reinscripción

- `POST /re-enrolments` (student)
- `GET /re-enrolments/mine` (student)
- `GET /re-enrolments` (admin)
- `POST /re-enrolments/{id}/submit` (student)
- `POST /re-enrolments/{id}/approve` (admin)
- `POST /re-enrolments/{id}/reject` (admin)
- `POST /re-enrolments/{id}/apply-to-moodle` (admin/worker)

## 5.6 Récord / documento oficial

- `POST /transcripts/generate` (student/admin según política)
- `GET /transcripts/mine` (student)
- `GET /transcripts/{transcriptNumber}` (verificación pública opcional)
- `POST /transcripts/{id}/revoke` (admin)

## 5.7 Integración Moodle (interna)

- `POST /integrations/moodle/webhook` (inbound desde Moodle)
- `POST /integrations/moodle/sync/bootstrap` (admin)
- `POST /integrations/moodle/sync/reconcile` (admin)
- `GET /integrations/moodle/health` (admin)
- `GET /integrations/moodle/status` (admin)

---

# 6) Permisos y accesos por rol (RBAC)

## 6.1 Matriz de permisos (resumen)

**Estudiantes**

- ☑ Ver calificaciones propias (`grades.read.own`)
- ☑ Ver reportes personales (`reports.read.own`)
- ☑ Solicitar reinscripción (`reenrol.create`, `reenrol.submit.own`)
- ☑ Descargar/generar récord (`transcript.generate.own`, `transcript.read.own`)
- ❌ Registrar/editar calificaciones
- ❌ Ver datos de otros estudiantes

**Profesores**

- ☑ Ver cursos asignados y estudiantes inscritos (`courses.read.assigned`, `students.read.assigned`)
- ☑ Registrar/editar calificaciones de sus cursos (`grades.write.assigned`)
- ☑ Generar reportes de sus cursos (`reports.read.assigned`)
- ❌ Procesar reinscripción global
- ❌ Gestión de usuarios / períodos

**Administradores**

- ☑ Gestión de usuarios/roles (`users.*`)

- ☑ Configurar períodos (`periods.*`)
- ☑ Procesar reinscripciones (`reenrol.*`)
- ☑ Reportes globales (`reports.read.all`)
- ☑ Reconciliación y monitoreo sync (`integration.*`)
- ☑ Generación/invalidación de récord oficial (`transcript.*`)

### 6.2 Reglas "contextuales"

- Un profesor solo puede escribir calificaciones si:
    1. está asignado al curso (mirror de enrolment/role), y
    2. el período está **abierto**, y
    3. el ítem de calificación es editable según política (p. ej. no cerrado).

---

# 7) Estrategia de sincronización de datos (bidireccional)

## 7.1 Clasificación: "quién manda" (Source of Truth por entidad)

Para minimizar conflictos, define autoridad:

- **Usuarios**: Moodle (o tu sistema de admisiones si existe). La app espeja y extiende.
- **Cursos / estructura**: Moodle.
- **Enrolments**: App inicia cambios por reinscripción, pero Moodle mantiene el estado final; la app reconcilia.
- **Calificaciones**:
    - Si el registro se hace en la app → la app es fuente para ese evento y empuja a Moodle.
    - Si el profesor edita en Moodle → Moodle es fuente y la app actualiza su espejo.
    - La app siempre conserva **auditoría** y "último estado conocido".

## 7.2 Mecanismos concretos de sincronización

### A) "Near real-time" por webhooks (recomendado)

- Instalar/configurar plugin de WebHooks para enviar POST a tu endpoint con eventos relevantes (enrolment creado/actualizado, usuario actualizado, curso actualizado, grade actualizado, actividad creada/cambiada). [moodle.org]
- Tu endpoint **no procesa pesado**: solo valida firma/secret, guarda evento y encola.

### B) Polling incremental (seguro y simple)

- Jobs cada N minutos para:
  - Reconsultar cambios recientes
  - Comparar `timemodified`
  - Reparar pérdidas de eventos
- Ideal como "seguro de vida" incluso con webhooks.

### C) Patrón Outbox (para outbound confiable)

- Cada acción que requiera "push" a Moodle:
  1. se guarda local (DB)
  2. se inserta evento en `outbox_events` **en la misma transacción**
  3. worker procesa con reintentos exponenciales
  4. idempotencia por clave única (course+item+user)

## 7.3 Resolución de conflictos (cuando cambia en ambos lados)

Política recomendada:

1. **Optimistic Concurrency + Last-Write-Wins controlado**
   - Guardar `timemodified` de Moodle (si lo obtienes por WS) y `updated_at` local
   - Si llega un cambio más viejo que el último aplicado → se ignora o se marca como conflicto
2. **Registro de conflicto**
   - Guardar en tabla `audit_log` o `sync_conflicts`
   - Notificar al admin si el conflicto afecta un récord oficial

## 7.4 Reconciliación

- Job nocturno (o semanal) que:
  - Selecciona cursos/períodos
  - Recalcula totales y compara con Moodle
  - Repara divergencias
- Reporte: "desviaciones" y acciones sugeridas

## 7.5 Seguridad de integración

#### En Moodle

- Habilitar Web Services, protocolo REST y crear token/servicio con solo funciones necesarias (principio de mínimo privilegio). [docs.moodle.org], [moodledev.io]
- Restringir por IP y expiración cuando aplique (mejor práctica indicada en la configuración de tokens/servicios). [docs.moodle.org]

**En la App**

- Endpoint webhook con:
  - Validación de secret compartido
  - Rate limiting
  - Registro de firma + replay protection (nonce/timestamp)
- Tokens Moodle guardados cifrados (KMS/KeyVault)

---

# 8) Funciones Moodle típicas a consumir (guía práctica)

Moodle recomienda crear **External Services** personalizados agregando solo las funciones necesarias y asignando capacidades requeridas. [docs.moodle.org], [moodledev.io]

A nivel conceptual, necesitarás WS para:

- **Cursos**: listar/obtener detalle, participantes
- **Usuarios**: listar/buscar, sincronizar estado
- **Enrolments**: matricular/suspender/consultar
- **Calificaciones**: obtener y actualizar (gradebook)
- **Actividades académicas**: consultar assignments/quizzes/grade items (si se requiere para reportes)

*(Los nombres exactos de `wsfunction` dependen de tu Moodle y de las funciones habilitadas; tu app debe configurarlo y probarlo en un ambiente staging.)*

---

# 9) Recomendaciones de implementación (para el equipo dev)

1. **Empieza por "Mirror mínimo viable"**: users, courses, enrolments, grade items, grades.
2. Implementa **Outbox + Workers** desde el día 1 (evita "doble escritura" sin consistencia).
3. Usa **webhooks** si el cliente permite instalar plugin; si no, "polling + reconciliación". [moodle.org]
4. Diseña con **idempotencia** (claves únicas, retries seguros).
5. Asegura que la lógica de reinscripción respete el concepto de enrolment vs role assignment (son cosas distintas en Moodle). [docs.moodle.org]

---

# 10) Preguntas clave para afinar la guía (solo para ajustar, no para frenar el diseño)

1. ¿La reinscripción debe **crear nuevos cursos** por período o reutiliza cursos con "grupos/cohortes"?

   No se colocaran en un espacio que permitira asignarlo al curso, pero el curso no se crea automatico

2. ¿Las calificaciones se registrarán **solo** en la app o también se permitirá edición en Moodle (y por quién)?

   Las calificaciones finales solo se regitraran en la app.

3. ¿El récord oficial requiere **firma digital**, sello institucional, o verificación pública (QR/URL)?

   Si